

xxxx 实验报告

姓名： 林宏宇 学号： 23320093

1. 实验内容

为处理器综合应用设计试验

2. 实验目的

1. 掌握单周期 CPU 的实现方法，代码实现方法；
2. 掌握测试单周期 CPU 的方法。
3. 输入输出原理；

3. 实验任务

改造 CPU，增加以下功能：

①寄存器堆调试接口改写：

增加 debug 接口，接收串口输入的调试地址 ra_debug，输出寄存器值 rd_debug 并通过串口显示，便于观察寄存器状态。

②接受外部输入：

引入串口作为外设，通过修改数据存储器访问逻辑，实现对串口寄存器状态的访问与操作。

③添加指令：

增加 sllv 和 srlv 指令，用于逻辑左移和逻辑右移操作，并扩展 ALU 解码器和控制信号。

④上板验证：

验证寄存器调试功能、外设读取功能及新增指令的正确性。

4. 实验过程

1. 单周期 CPU 仿真通过

保证基本指令功能如 add、lw、sw 的正确性，通过仿真验证 ALU、寄存器堆和控制器的协同工作。

2. 增加调试接口

- 在寄存器堆 (regfile.v) 中增加 ra_debug 和 rd_debug 接口，并连接至顶层模块。
- 实现通过串口接收调试地址并返回寄存器值的功能。

```
24 module regfile(  
25     input wire      clk_i,  
26     input wire      we3_i,  
27     input wire      we_dbg_i,  
28     input wire [4:0] ra1_i,  
29     input wire [4:0] ra2_i,  
30     input wire [4:0] ra_dbg_i,  
31     input wire [4:0] wa3_i,  
32     input wire [31:0] wd3_i,  
33     input wire [4:0] wa_dbg_i,  
34     input wire [31:0] wd_dbg_i,  
35     output wire [31:0] rd1_o,  
36     output wire [31:0] rd2_o,  
37     output wire [31:0] rd_dbg_o  
38 );  
39     reg [31:0] rf[31:0];  
40     wire [4:0] wr_addr;  
41     wire [31:0] wr_data;  
42     assign wr_addr = we_dbg_i ? wa_dbg_i : wa3_i;  
43     assign wr_data = we_dbg_i ? wd_dbg_i : wd3_i;  
44     always @(posedge clk_i) begin  
45         if(we3_i | we_dbg_i) begin  
46             rf[wr_addr] <= wr_data;  
47         end  
48     end  
49     assign rd1_o = (ra1_i != 0) ? rf[ra1_i] : 0;  
50     assign rd2_o = (ra2_i != 0) ? rf[ra2_i] : 0;  
51     assign rd_dbg_o = (ra_dbg_i != 0) ? rf[ra_dbg_i] : 0;  
52 endmodule
```

3. 接收外部输入

添加串口寄存器，保存串口接收到的数据，用于与 CPU 的交互。

```

mips u_mips(
    .clk_i      (clk_i      ),
    .rst_n      (rst_n      ),
    .pc_run_en_i (pc_run_en_i ),
    .pc_clr_i    (pc_clr_i    ),
    .pc_o        (pc        ),
    .instr_i     (instr      ),
    .memwrite_o   (memwrite   ),
    .aluout_o     (dataaddr   ),
    .writedata_o  (writedata  ),
    .readdata_i   (readdata   ),
    .monitor_list_o (monitor_list_o ),
    .ra_debug_i   (ra_debug_i ),
    .ra_debug_data_o (ra_debug_data_o ),
    .reg_wdata_dbg (reg_wdata_dbg ),
    .reg_wren_dbg  (reg_wren_dbg )
);

inst_rom u_inst_rom (
    .clka(clk_i),    // input wire clka
    .ena(1'b1),      // input wire ena
    .addra(pc[9:2]), // input wire [3 : 0] addra
    .douta(instr)    // output wire [31 : 0] douta
);

data_ram u_data_ram (
    .clka(~clk_i),    // input wire clka
    .ena(1'b1),      // input wire ena
    .wea(memwrite),   // input wire [0 : 0] wea
    .addra(memaddr_actual[13:0]), // input wire [13 : 0] addra
    .dina(memwdata_actual), // input wire [31 : 0] dina

data_ram u_data_ram (
    .clka(~clk_i),    // input wire clka
    .ena(1'b1),      // input wire ena
    .wea(memwrite),   // input wire [0 : 0] wea
    .addra(memaddr_actual[13:0]), // input wire [13 : 0] addra
    .dina(memwdata_actual), // input wire [31 : 0] dina
    .douta(memrdata) // output wire [31 : 0] douta
);

assign memaddr_actual = mem_debug_rden_i ? mem_debug_addr_i : memaddr;
assign memwdata_actual = writedata;
assign memrdata_actual = (memaddr_actual[14]) ? uart_reg_i : memrdata;

assign memwdata = writedata;
assign memaddr = dataaddr[14:0];
assign mem_debug_rdata_o = memrdata_actual;
assign readdata = memrdata_actual;

assign pc_o = pc;
assign instr_o = instr;

endmodule

```

4. 添加新指令

在 ALU 解码器中增加 `sllv` 和 `srlv` 指令的控制信号，同时 `alucontrol` 扩展编码位宽。

```

wire[31:0] s,bout;
reg[31:0] y;
assign result_o=y;
assign bout = op_i[2] ? ~B_i : B_i;
assign s = A_i + bout + op_i[2];
always @(*) begin
    case (op_i[2:0])
        3'b000: y <= A_i & bout;
        3'b001: y <= A_i | bout;
        3'b010: y <= s;
        3'b110: y <= s;
        3'b111: y <= s[31];
        3'b100: y <=A_i<<B_i;
        3'b101: y <=A_i>>B_i;
        default : y <= 32'b0;
    endcase
end
assign zero_o = (y == 32'b0);

```

```

module aludec(
    input    [5:0]  funct_i,
    input    [1:0]  aluop_i,
    output reg [2:0] alucontrol_o
);
always@(*)begin
    case(aluop_i)
        2'b00:begin
            alucontrol_o<=3'b010;
        end
        2'b01:begin
            alucontrol_o<=3'b110;
        end
        2'b10:begin
            case(funct_i)
                6'b000000:alucontrol_o<=3'b100;
                6'b000010:alucontrol_o<=3'b101;
                6'b100000:alucontrol_o<=3'b010;
                6'b100010:alucontrol_o<=3'b110;
                6'b100100:alucontrol_o<=3'b000;
                6'b100101:alucontrol_o<=3'b001;
                6'b101010:alucontrol_o<=3'b111;
                default:
                    alucontrol_o<=3'b0;
            endcase
        end
        default:
            alucontrol_o<=3'b0;
    endcase
end
end

```

5. 上板验证

1.在 mars 编写代码，并且产生相应的二进制 coe 文件

```
mipstest.coe  data (10).txt  data (11).txt  data (12).txt  m
文件  编辑  查看

memory initialization radix = 16;
memory initialization vector =
8c024000,
200d7fff,
004d5824,
200e000f,
004e8002,
020d5024,
020e8802,
200d0003,
022d4824,
11200006,
20010001,
10290007,
20010002,
10290008,
20010003,
1029000e,
014b2020,
20080000,
08000000,
014b2022,
20080000,
08000000,
00002020,
1160ffe8,
008a7820,
01e02020,
20010001,
01617822,
01e05820,
08000017,
00002020,
00006020,
000a602a,
1180ffde,
014b7822,
01e05020,
208f0001,
01e02020,
08000020

行 1, 列 1  | 454 个字符
```

2.二进制指令

CPU 可以读

取 32 位串口状态寄存器的值，其中 31-30 位为计算器的功能位，其中功能位的定义如下：

00 01 10 11

加法 减法 乘法 除法

其中，乘法和除法运算需要使用加减法实现(复用 mars 实验代码)

剩下的 30 位（平均分为 15 位）分别为两个输入数；要求根据 31-30 位寄存器

的值调用不同的子函数，然后将最后的计算结果放在\$4 和\$8 里面

3. 串口指令码

1. 串口指令码总览

功能说明：串口指令是用于与串口监视器交互的指令，而不是单周期 CPU 的指令。

指令代码及功能：

00000000：运行

00000001：复位

00000100：寄存器操作

00000110：单步执行

00000101：数据内存操作

2. 寄存器操作

指令代码及功能说明：

00000100 010 xxxxx：查看某个寄存器的值。后续 xxxxx 为寄存器地址范围（0~31）。

00000100 110 00000：查看全部寄存器的值，地址字段无效（推荐填入 00000）。

00000100 100 xxxxx dddddddd：向某个寄存器写入值，其中 xxxxx 为寄存器地址，dddddddd 为 32 位数据。

00000100 111 00000 dddddddd：向串口寄存器写入值，地址字段无效。

3. 内存操作

指令代码及功能说明：

00000101 0 xxxxxxxxxxxxxx：查看某个地址的内存值，其中 xxxxxxxxxxxxxx 为 14 位内存地址。

6. 实验结果

```
[2025-01-12 13:23:10.646]
TX: 01
[2025-01-12 13:23:27.338]
TX: 04e080018003
[2025-01-12 13:23:31.598]
TX: 00
[2025-01-12 13:23:32.617]
RX: 35 40 00 00 35 40 00 04 35 40 00 08 35 40 00 0C 35 40 00 10 35 40 00 14 35 40 00
18 35 40 00 1C
[2025-01-12 13:23:37.121]
TX: 0448
[2025-01-12 13:23:38.138]
RX: 00 00 00 09
```

输入 01：复位

输入 04e080018003：根据上面的二进制内容和串口指令，此指令为乘法 3*3

输入 00：启动

输入 0448：查看第八号寄存器

输出 00 00 00 09：3*3 的结果为 9

串口测试验证上板测试结果正确

7. 总结

本实验围绕单周期 CPU 的功能扩展展开，完成了寄存器调试接口改造、串口外设交互、新指令添加及上板验证任务。

功能实现：1.寄存器调试接口：通过增加 ra_debug 和 rd_debug 接口，支持串口输入调试地址并输出寄存器值，便于实时观察寄存器状态。2.外设输入功能：引入串口寄存器，实现对外部输入数据的保存和访问，并通过存储器映射机制区分存储器与外设访问区域。3.新增指令：在 ALU 解码器中添加 slv 和 srlv 指令，扩展位宽支持逻辑左移和右移操作。

测试验证：通过仿真验证基本指令与新增功能的正确性，结合 Mars 生成的二进制程序实现串口交互操作，并成功在开发板上完成功能测试，验证了设计的可行性和正确性，全面掌握了单周期 CPU 的实现与调试方法。